

UNIVERSIDADE FEDERAL DO ABC

BEATRIZ DOMINGOS DE OLIVEIRA, RA 11202130893

PROJETO DE PROGRAMAÇÃO - KV STORE

Santo André, São Paulo.

2026

BEATRIZ DOMINGOS DE OLIVEIRA, RA 11202130893

PROJETO DE PROGRAMAÇÃO - KV STORE

Trabalho apresentado ao Curso de Sistemas Distribuídos como parte dos requisitos necessários para conclusão da disciplina.

Prof. Dr. Vladimir Moreira Roch

Santo André, São Paulo

2026

1 INTRODUÇÃO **3**
2 ESTRUTURA DO PROJETO **4**
Visão Geral: 4
Funcionalidades e Estrutura de Dados do Servidor : 4
Inicialização e atendimento concorrente (linhas 60–87; 144–193) 4
PUT recebido por servidor não líder (linhas 332–348) 4
PUT recebido pelo líder (linhas 195–233) 5
Replicação e confirmação — REPLICATION e REPLICATION_OK (linhas 235–330; 350–393) 5
Processamento de GET (linhas 505–562) 6
Entrega assíncrona de GET pendente (linhas 395–464) 6
Recuperação de servidor não líder (linhas 96–142; 466–503) 7
Estrutura de dados 7
Funcionalidades e Estrutura de Dados do Cliente: 8
Inicialização e menu interativo (linhas 43–92) 8
Escolha do servidor (linhas 166–201) 9
Envio de PUT (linhas 94–115) 9
Envio de GET (linhas 117–150) 10
Recebimento de respostas assíncronas e gerenciamento de timestamps (linhas 152–164; 203–244) 10
Estrutura de Dados Cliente 11
Implementação de threads 11
Threads no servidor (linhas 82–86; 144–193; 279–330; 437–464) 11
Threads no cliente (linhas 43–70; 215–244) 12
Proteção contra acessos concorrentes 12
Proteção das estruturas compartilhadas no servidor 12
Proteção dos timestamps no cliente 14
3 RESULTADOS **15**
4 DISCUSSÃO **15**
5 REFERÊNCIAS **16**

1 INTRODUÇÃO

Este trabalho tem por objetivo demonstrar de forma prática conceitos fundamentais estudados na disciplina de Sistemas Distribuídos, como comunicação entre processos, replicação de dados, consistência, concorrência e tolerância a falhas.

Para isso, foi desenvolvido um sistema distribuído de armazenamento chave-valor (*Key-Value Store*), composto por três servidores e múltiplos clientes, que se comunicam por meio do protocolo TCP. A implementação utiliza uma arquitetura com líder responsável pelas operações de escrita, replicação entre servidores e mecanismos de sincronização que garantem que os clientes não recebam versões anteriores dos dados já consultados, além de permitir a recuperação de informações após a reinicialização de um servidor.

2 ESTRUTURA DO PROJETO

Visão Geral:

O sistema implementa uma KV Store replicada com três servidores TCP, um líder fixo e múltiplos clientes. O cliente seleciona aleatoriamente um servidor para PUT ou GET, sem conhecer o líder. Os PUTs são processados pelo líder e replicados; os GETs são atendidos localmente, respeitando o timestamp já observado pelo cliente.

Foram implementadas somente as classes Servidor, Cliente e Mensagem (que serão melhor explicadas abaixo); as classes de threads são aninhadas. A comunicação transfere objetos Mensagem codificados em JSON com Gson 2.10.1, biblioteca admitida no enunciado do projeto.

Funcionalidades e Estrutura de Dados do Servidor :

Inicialização e atendimento concorrente (linhas 60–87; 144–193)

O processo de inicialização do servidor foi desenvolvido para que o usuário informe o IP e a porta do servidor local, além do IP e da porta do líder (linhas 61–71). Em seguida, é criado um ServerSocket (linha 75), responsável por receber as conexões TCP. Caso o servidor não seja o líder, ele solicita uma sincronização inicial para obter os dados já armazenados (linhas 77–80).

Após a inicialização, o servidor permanece aguardando conexões por meio de um laço accept() (linhas 82–86). Para cada nova conexão, é criada uma ThreadAtendimento, permitindo atender múltiplas requisições simultaneamente. Cada thread recebe um objeto Mensagem (linha 161), identifica o tipo da requisição para executar o tratamento correspondente (linhas 167–179) e encerra a conexão após o atendimento (linhas 180–183).

PUT recebido por servidor não líder (linhas 332–348)

Quando um servidor não líder recebe uma requisição PUT, ele não realiza a escrita diretamente, pois essa responsabilidade é exclusiva do líder. Nesse caso, o servidor registra o encaminhamento no console (linha 336), estabelece uma conexão TCP com o líder (linha 338) e repassa a mensagem recebida, preservando as informações de identificação do cliente original (linha 344).

Após o processamento pelo líder, o servidor intermediário recebe a confirmação PUT_OK e a encaminha ao cliente pela conexão originalmente estabelecida (linha 345).

PUT recebido pelo líder (linhas 195–233)

Quando o líder recebe uma requisição PUT, ele registra no console as informações enviadas pelo cliente (linhas 203–204). Em seguida, acessa uma região crítica protegida por sincronização para consultar o timestamp atual da chave, incrementá-lo e atualizar o valor armazenado (linhas 209–218).

O timestamp funciona como um contador lógico individual para cada chave, permitindo identificar a versão mais recente de um dado. Caso a chave já exista, tanto seu valor quanto seu timestamp são atualizados.

Após a escrita local, o líder inicia o processo de replicação para os demais servidores (linha 222). Somente após aguardar as confirmações dos servidores participantes, o líder envia ao cliente a mensagem PUT_OK, acompanhada do timestamp atribuído à operação (linhas 224–232).

Replicação e confirmação — REPLICATION e REPLICATION_OK (linhas 235–330; 350–393)

O mecanismo de replicação foi implementado para manter as informações armazenadas nos três servidores atualizadas. Após processar um PUT, o líder cria uma mensagem REPLICATION, contendo a chave, o valor e o timestamp da atualização (linhas 239–243).

Para cada servidor registrado como ativo, o líder cria uma ThreadReplicacao (linhas 247–259). Dessa forma, as mensagens de replicação são enviadas em paralelo, sem que seja necessário aguardar a resposta de um servidor para iniciar a comunicação com outro.

Cada thread aguarda o recebimento de uma mensagem REPLICATION_OK (linhas 302–313). O líder utiliza o método join() para aguardar a conclusão das threads (linhas 262–266). Caso algum servidor não confirme a replicação, ele é marcado como inativo (linhas 267–275).

Ao receber uma mensagem REPLICATION, o servidor não líder registra a operação no console (linha 356) e compara o timestamp recebido com o armazenado localmente. A atualização é aplicada somente quando o timestamp

recebido é superior ao atual (linhas 370–378), evitando que mensagens atrasadas sobrescrevam versões mais recentes.

Após atualizar sua tabela, o servidor verifica se existem requisições GET pendentes que agora podem ser atendidas (linhas 379–386) e envia a confirmação REPLICATION_OK ao líder (linhas 388–392).

Processamento de GET (linhas 505–562)

O processamento das requisições GET pode ser realizado por qualquer servidor, independentemente de ele ser o líder. Para garantir que o cliente não receba uma versão anterior àquela que já conhece, cada requisição contém o último timestamp associado à chave no cliente.

Ao receber a solicitação, o servidor acessa uma região crítica e compara o timestamp informado pelo cliente com o timestamp armazenado localmente (linhas 523–551).

A partir dessa comparação, são considerados três cenários:

- **Chave inexistente e timestamp do cliente igual a zero:** o servidor retorna NULL e timestamp zero, indicando que não possui o valor solicitado (linhas 531–535).
- **Timestamp local maior ou igual ao do cliente:** o servidor retorna o valor armazenado e seu timestamp, pois a versão disponível atende ao requisito de consistência (linhas 536–540).
- **Timestamp local inferior ao do cliente:** o servidor responde WAIT_FOR_RESPONSE e registra a requisição em uma lista de pedidos pendentes (linhas 541–549).

No último caso, a conexão original é encerrada, permitindo que o cliente continue executando outras operações enquanto aguarda a atualização do servidor.

Entrega assíncrona de GET pendente (linhas 395–464)

A entrega assíncrona foi implementada para atender requisições GET que inicialmente não puderam ser respondidas porque o servidor possuía uma versão desatualizada da chave.

Quando uma atualização é recebida, o método retirarPendentesAtendidos percorre os pedidos pendentes e identifica aqueles cujo timestamp solicitado já foi

alcançado. Esses pedidos são removidos da lista e suas respectivas respostas são preparadas (linhas 395–427).

Em seguida, o método entregarPendentes cria uma ThreadEntregaPendentes para cada resposta (linhas 429–464). Cada thread estabelece uma nova conexão TCP com a porta de escuta do cliente e envia o valor atualizado.

Dessa forma, o sistema utiliza uma comunicação assíncrona orientada a eventos, na qual a resposta definitiva é enviada quando a atualização necessária fica disponível, sem manter a conexão original aberta.

Recuperação de servidor não líder (linhas 96–142; 466–503)

O mecanismo de recuperação foi desenvolvido para permitir que um servidor não líder volte a participar do sistema após uma queda, mesmo que tenha perdido todas as informações armazenadas em memória.

Durante sua inicialização, o servidor estabelece uma conexão com o líder e envia uma mensagem SYNC_REQUEST, solicitando uma cópia dos dados atuais (linhas 110–114). Caso a comunicação não seja estabelecida, novas tentativas são realizadas (linhas 134–140).

Ao receber a solicitação, o líder registra o servidor e prepara uma mensagem SYNC_RESPONSE, contendo cópias das tabelas de valores e timestamps armazenadas (linhas 466–503, especialmente 483–494).

O servidor não líder recebe a resposta e restaura suas tabelas locais (linhas 119–127), recuperando o estado conhecido pelo líder no momento da sincronização.

Quando se trata de um servidor previamente registrado que está retornando após uma queda, o sistema também apresenta no console a mensagem de confirmação da recuperação (linhas 128–131).

Estrutura de dados

Cada servidor guarda os dados em duas tabelas hash: tabelaValores (linha 40), que associa cada chave ao seu valor, e tabelaTimestamps (42), que associa cada chave à sua versão lógica, ou seja, quantas vezes ela já foi atualizada. Duas estruturas são usadas só pelo líder. registroServidores (46) guarda o endereço de cada servidor e se ele está ativo ou inativo, para que a replicação seja enviada

apenas aos ativos. numeroRegistros (49) conta quantas vezes cada servidor já se registrou, para que uma replicação que falhou contra um servidor que caiu não marque como inativo esse mesmo servidor quando ele acabar de voltar. Os GETs que receberam WAIT_FOR_RESPONSE ficam em getsPendentes (53), uma lista de Mensagem em que cada item guarda a chave, o timestamp do cliente e o endereço de escuta dele. Todas essas estruturas são compartilhadas entre as threads e protegidas por uma única trava, travaEstado (58).

O servidor também usa algumas estruturas temporárias, que existem só durante um pedido. Durante um PUT, o líder monta a lista threads (247) com uma ThreadReplicacao para cada servidor ativo e a usa para esperar todas com join(). Ao aplicar uma replicação, a lista entregas (369 e 400) reúne as respostas já prontas dos GETs pendentes que a nova versão atende, para que sejam enviadas fora da trava. No atendimento de um SYNC_REQUEST, copiaValores e copiaTimestamps (480–481) guardam uma cópia da tabela tirada dentro da trava, e é essa cópia que vai no SYNC_RESPONSE, sem bloquear os outros pedidos durante o envio.

Funcionalidades e Estrutura de Dados do Cliente:

Inicialização e menu interativo (linhas 43–92)

O cliente possui um menu interativo que permite ao usuário escolher entre as operações INIT, PUT e GET (linhas 43–70). Para garantir que as requisições sejam realizadas somente após a configuração dos servidores, o programa impede a execução de PUT e GET antes da inicialização (linhas 57–58).

Ao selecionar INIT, o usuário informa os endereços IP e as portas dos três servidores disponíveis (linhas 75–80). Em seguida, é criado um ServerSocket(0), que disponibiliza uma porta TCP livre para o recebimento de conexões. Essa porta é utilizada posteriormente para receber respostas assíncronas dos servidores.

Por fim, é iniciada uma ThreadEscuta (linhas 83–91), responsável por monitorar a chegada dessas respostas sem interromper a interação do usuário com o menu principal.

Escolha do servidor (linhas 166–201)

O processo de escolha do servidor foi desenvolvido para distribuir as requisições entre os três servidores disponíveis, sem que o cliente precise conhecer qual deles é o líder.

A cada operação, o programa seleciona aleatoriamente um servidor como destino inicial (linha 172). Caso a tentativa de conexão falhe, o cliente procura estabelecer comunicação com os demais servidores por meio de um mecanismo de rodízio (linhas 174–175).

Antes do envio da requisição, são adicionadas à mensagem as informações de IP e porta de retorno do cliente (linhas 184–185). Essas informações permitem que o servidor estabeleça uma nova conexão com o cliente caso seja necessário enviar uma resposta assíncrona.

O cliente também registra qual servidor foi consultado (linhas 189–193). Se nenhuma das tentativas de conexão for bem-sucedida, o programa informa a indisponibilidade dos servidores (linha 199).

Envio de PUT (linhas 94–115)

A operação PUT permite ao cliente inserir ou atualizar um par chave-valor no sistema distribuído.

Ao selecionar essa opção, o usuário informa a chave (key) e o valor (value) que deseja armazenar (linhas 97–100). Com essas informações, o cliente cria um objeto Mensagem do tipo PUT (linhas 102–105) e o encaminha a um dos servidores disponíveis (linha 107).

Após o processamento da requisição e a confirmação da replicação pelo líder, o cliente recebe uma mensagem PUT_OK, contendo o timestamp associado à atualização.

Esse timestamp é utilizado para atualizar o registro local da versão conhecida pelo cliente (linha 109). Por fim, o programa apresenta no console a confirmação da operação, incluindo a chave, o valor, o timestamp e o endereço do servidor responsável pelo atendimento (linhas 110–113).

Envio de GET (linhas 117–150)

A operação GET permite consultar o valor associado a uma chave armazenada no sistema. Diferentemente do PUT, o usuário precisa informar apenas a chave desejada (linhas 120–121).

Antes de enviar a requisição, o cliente consulta sua estrutura local de timestamps para identificar a versão mais recente que já conhece daquela chave. Caso não exista um registro anterior, o timestamp utilizado será zero (linhas 123–128).

Em seguida, é criada uma mensagem GET, contendo a chave solicitada e o timestamp conhecido pelo cliente (linhas 130–133). Essas informações permitem que o servidor verifique se possui uma versão suficientemente atualizada para responder à solicitação.

Quando o servidor possui uma versão igual ou mais recente, o cliente recebe uma mensagem GET_RESPONSE e apresenta o valor retornado no console (linhas 141–143).

Caso o servidor esteja desatualizado, a resposta será WAIT_FOR_RESPONSE. Nessa situação, o cliente informa que a resposta está pendente e retorna ao menu principal, sem manter a conexão original aberta (linhas 144–149).

Dessa forma, o usuário pode continuar realizando outras operações enquanto aguarda a atualização necessária para concluir a consulta.

Recebimento de respostas assíncronas e gerenciamento de timestamps (linhas 152–164; 203–244)

O recebimento de respostas assíncronas foi implementado para permitir que o cliente obtenha o resultado de um GET mesmo após o encerramento da conexão original com o servidor.

Para isso, a ThreadEscuta permanece aguardando novas conexões TCP na porta disponibilizada durante a inicialização. Quando um servidor envia uma resposta pendente, essa thread aceita a conexão, recebe o objeto Mensagem e encaminha seu processamento (linhas 215–244).

O tratamento da resposta é realizado pelo método tratarRespostaGet (linhas 203–213), que permite apresentar o valor recebido sem interromper a execução do menu principal.

Além disso, o cliente mantém uma estrutura local que associa cada chave ao maior timestamp já observado. O método atualizarTimestamp compara o timestamp recebido com o valor armazenado e preserva sempre o maior deles (linhas 155–163).

Esse mecanismo é fundamental para a consistência das leituras, pois impede que o cliente aceite versões anteriores às que já conhece, mesmo quando suas requisições são atendidas por servidores diferentes.

Estrutura de Dados Cliente

No INIT, o cliente guarda o IP e a porta dos três servidores em dois vetores, ipsServidores (Cliente.java, 24) e portasServidores (26), e marca inicializado (28) como verdadeiro, o que libera as opções PUT e GET do menu. O maior timestamp que o cliente já viu para cada chave fica em timestampsPorKey (31), que começa em zero para toda chave e é enviado em cada GET. Como o menu e a thread de escuta podem acessar essa tabela ao mesmo tempo, ela é protegida por travaTimestamps (33). As respostas assíncronas chegam pelo servidorEscuta (36), um ServerSocket aberto numa porta escolhida pelo sistema operacional. Essa porta fica em portaEscuta (38) e vai em cada GET, para o servidor saber onde responder depois. Por fim, sorteio (41) escolhe aleatoriamente o primeiro servidor de cada

Implementação de threads

Threads no servidor (linhas 82–86; 144–193; 279–330; 437–464)

O uso de threads no servidor foi implementado para permitir o atendimento simultâneo de múltiplas requisições, a replicação paralela dos dados e o envio de respostas assíncronas, sem que uma dessas operações impeça a execução das demais.

A thread principal do servidor permanece aguardando novas conexões TCP por meio do método accept() (linhas 82–86). Sempre que uma conexão é estabelecida, é criada uma ThreadAtendimento (linha 85), responsável por receber a mensagem, identificar o tipo da requisição e executar seu processamento de forma independente (linhas 144–193). Dessa forma, o servidor pode continuar recebendo conexões enquanto outras requisições estão sendo atendidas.

Para realizar a replicação dos dados, o líder utiliza a classe ThreadReplicacao (linhas 279–330). Uma instância dessa thread é criada para cada servidor ativo (linha 251), permitindo que as mensagens REPLICATION sejam enviadas paralelamente (linhas 257–259). Cada thread aguarda a confirmação REPLICATION_OK do servidor correspondente. O líder utiliza o método join() para aguardar a conclusão dessas operações (linhas 262–266) antes de prosseguir com a confirmação da escrita ao cliente.

Além disso, a classe ThreadEntregaPendentes (linhas 437–464) é utilizada para enviar respostas de requisições GET que anteriormente receberam WAIT_FOR_RESPONSE. Quando a versão solicitada fica disponível, uma thread é criada (linha 433) para estabelecer uma nova conexão com o cliente e enviar o resultado, sem bloquear a thread principal responsável pelo recebimento de conexões.

Threads no cliente (linhas 43–70; 215–244)

No cliente, o uso de threads permite que o usuário continue interagindo com o sistema enquanto aguarda respostas assíncronas dos servidores.

A thread principal é responsável pela execução do menu interativo e pelo envio das operações INIT, PUT e GET (linhas 43–70).

Durante a inicialização, é criada e iniciada uma ThreadEscuta (linha 87), responsável por aguardar conexões TCP na porta disponibilizada pelo cliente. Essa thread permanece executando independentemente do menu principal e processa as mensagens recebidas dos servidores (linhas 215–244).

Essa separação permite que o cliente receba posteriormente o resultado de uma requisição GET que inicialmente retornou WAIT_FOR_RESPONSE, sem precisar interromper a interação do usuário ou manter a conexão original aberta.

Proteção contra acessos concorrentes

Proteção das estruturas compartilhadas no servidor

Como o servidor utiliza múltiplas threads para atender requisições simultaneamente, diferentes operações podem tentar acessar ou modificar as mesmas estruturas de dados. Para evitar condições de corrida e preservar a consistência das informações, foi implementado um mecanismo de exclusão mútua

utilizando synchronized, sem recorrer a estruturas concorrentes prontas da linguagem Java.

O controle é realizado por meio do objeto travaEstado, declarado na linha 58 de Servidor.java. Sempre que uma operação precisa acessar estruturas compartilhadas de maneira consistente, ela executa o trecho correspondente dentro de um bloco synchronized(travaEstado).

Esse mecanismo é utilizado em diferentes situações:

- Atualização de valores e timestamps: durante o processamento de um PUT, a alteração do valor e o incremento do timestamp são realizados dentro da mesma região crítica (linhas 209–218), impedindo que duas threads modifiquem simultaneamente a mesma chave.
- Gerenciamento dos servidores ativos: a consulta aos servidores disponíveis para replicação (linhas 248–254) e a atualização do estado de servidores que não confirmaram uma operação (linhas 268–274) são protegidas para evitar inconsistências no registro de participantes.
- Recebimento de replicações: a atualização das tabelas locais e a identificação das requisições GET pendentes que podem ser atendidas ocorrem de forma sincronizada (linhas 370–382), garantindo que essas operações considerem um estado consistente dos dados.
- Sincronização de servidores: durante a recuperação de um servidor não líder, o registro do participante e a criação das cópias das tabelas são realizados sob proteção (linhas 483–494), evitando alterações concorrentes durante a obtenção do estado.
- Processamento de GET: a comparação dos timestamps e o registro de requisições pendentes são protegidos pela mesma região crítica (linhas 523–551). Isso impede que uma atualização seja realizada entre a identificação de uma versão desatualizada e o registro do pedido na lista de espera.

As operações de comunicação pela rede são realizadas fora das regiões críticas, evitando que a espera por respostas TCP mantenha as estruturas compartilhadas bloqueadas por períodos prolongados.

Proteção dos timestamps no cliente

No cliente, também existe a possibilidade de acesso concorrente aos dados, pois a thread principal e a ThreadEscuta podem consultar ou atualizar os timestamps associados às chaves.

Para controlar esse acesso, foi criado o objeto travaTimestamps, declarado na linha 33 de Cliente.java, utilizado em blocos synchronized(travaTimestamps).

Esse mecanismo protege tanto a consulta do timestamp que será enviado em uma requisição GET (linhas 124–128) quanto a atualização do maior timestamp conhecido pelo cliente (linhas 155–163).

Durante a atualização, o programa compara o timestamp recebido com o valor já armazenado e mantém sempre o maior deles. Assim, mesmo que respostas sejam recebidas por threads diferentes, uma versão mais antiga não substitui uma versão mais recente.

Dessa forma, a sincronização no cliente contribui para preservar a consistência das leituras, enquanto a sincronização no servidor protege a integridade das estruturas compartilhadas durante o processamento concorrente das requisições.

3 RESULTADOS

Como resultado do projeto, foram desenvolvidos os programas cliente e servidor, responsáveis pela comunicação TCP, armazenamento e replicação de dados e manutenção da consistência entre os servidores. O código-fonte completo está disponível no repositório do GitHub, enquanto o vídeo de demonstração apresenta a compilação e a execução do sistema, incluindo operações PUT e GET, o tratamento assíncrono de WAIT_FOR_RESPONSE e a recuperação dos dados após a reinicialização de um servidor não líder.

4 DISCUSSÃO

Embora o sistema desenvolvido cumpra sua função em demonstrar a implementação de um sistema distribuído, algumas limitações tornam-o distante de um sistema real, e poderiam ser exploradas em futuras implementações para ampliar sua robustez e consistência. Atualmente, o líder é definido de forma fixa, o que representa um ponto único de falha para as operações de escrita. Como melhoria, poderia ser implementado um mecanismo de eleição de líder, permitindo a continuidade das operações em caso de indisponibilidade. Além disso, a inclusão de mecanismos de detecção de falhas, retransmissão de mensagens e sincronização automática das réplicas permitiria lidar melhor com interrupções de comunicação e servidores temporariamente indisponíveis. Por fim, a persistência dos dados em armazenamento não volátil reduziria a dependência da recuperação pelo líder após reinicializações, aproximando o projeto do funcionamento de sistemas distribuídos utilizados em ambientes reais.

5 REFERÊNCIAS

APACHE SOFTWARE FOUNDATION. ZooKeeper Programmer’s Guide. https://zookeeper.apache.org/doc/current/zookeeperProgrammers.html (acesso em 02 nov. 2026).

VAN STEEN, Maarten; TANENBAUM, Andrew S. Distributed Systems. 4. ed. https://www.distributed-systems.net.

ORACLE. The Java Tutorials: Writing the Server Side of a Socket. https://docs.oracle.com/javase/tutorial/networking/sockets/clientServer.html (acesso em 02 nov. 2026).

GOOGLE. Gson. https://github.com/google/gson (acesso em 02 nov. 2026).

ANTHROPIC. Claude, assistente de IA. https://claude.ai (acesso em 02 nov. 2026). Utilizado para apoiar depuração, revisão de concorrência e Java 8, além da revisão do relatório.